

AI Practical Manual

Develop a Rule-Based Expert System using Forward Chaining and Backward Chaining Inference Mechanisms

1. Aim

To develop a Rule-Based Expert System using Forward Chaining and Backward Chaining inference mechanisms in Python.

2. Objectives

1. Understand Rule-Based Expert Systems.
2. Understand facts, rules and inference engines.
3. Implement Forward Chaining in Python.
4. Implement Backward Chaining in Python.
5. Compare both inference mechanisms.

3. Theory

An Expert System is an Artificial Intelligence system that uses stored knowledge and reasoning techniques to solve problems or make decisions similar to a human expert.

A Rule-Based Expert System represents knowledge using **Facts** and **IF-THEN Rules**. The inference engine applies these rules to available facts and derives conclusions.

4. Components of a Rule-Based Expert System

Knowledge Base: Stores facts and rules.

Inference Engine: Applies rules and derives conclusions.

Facts: Information already known to be true.

Rules: Knowledge represented in IF-THEN form.

5. Forward Chaining

Forward Chaining is a **data-driven** inference mechanism. It starts with known facts and applies rules to generate new facts until a conclusion is reached.

Example: If the facts are **fever** and **cough**, and the rule is **IF fever AND cough THEN flu**, the system derives **flu**.

6. Backward Chaining

Backward Chaining is a **goal-driven** inference mechanism. It starts with a goal or conclusion and works backward to determine whether the required facts are available.

Example: For goal **flu**, the system searches for a rule that concludes flu, then checks whether fever and cough are available.

7. Difference Between Forward and Backward Chaining

Forward Chaining	Backward Chaining
------------------	-------------------

Data-driven	Goal-driven
Starts with facts	Starts with a goal
Moves from facts to conclusion	Moves from conclusion to facts
Useful when many facts are available	Useful for verifying a specific goal

8. Knowledge Base Used

Initial facts: **fever, cough**

Rule 1: IF fever AND cough THEN flu

Rule 2: IF flu AND body_pain THEN viral_infection

Rule 3: IF fever AND headache THEN infection

Main goal: **flu**

9. Forward Chaining Algorithm

1. Start with initial facts.
2. Check each rule.
3. Check whether all conditions of a rule are present in the facts.
4. If conditions are satisfied, derive the conclusion.
5. Add the conclusion to the facts.
6. Repeat until no new fact can be generated.
7. Display the derived conclusion.

10. Backward Chaining Algorithm

1. Select a goal.
2. Check whether the goal is already a known fact.
3. Search for a rule whose conclusion matches the goal.
4. Treat the rule conditions as sub-goals.
5. Check whether each sub-goal can be proved.
6. If all sub-goals are proved, the goal is proved.
7. Otherwise, the goal cannot be proved.

11. Python Program

```
# Rule-Based Expert System
# Forward Chaining and Backward Chaining

facts = {
    "fever",
    "cough"
}

rules = [
    {
        "conditions": ["fever", "cough"],
        "conclusion": "flu"
    },
    {
        "conditions": ["flu", "body_pain"],
        "conclusion": "viral_infection"
    },
    {
        "conditions": ["fever", "headache"],
        "conclusion": "infection"
    }
]

# -----
```

```

# Forward Chaining
# -----
def forward_chaining(facts, rules):
    facts = set(facts)
    print("\nFORWARD CHAINING")
    print("-----")

    changed = True
    while changed:
        changed = False

        for rule in rules:
            conditions = rule["conditions"]
            conclusion = rule["conclusion"]

            if all(condition in facts for condition in conditions):
                if conclusion not in facts:
                    print("Rule Fired: IF",
                        " AND ".join(conditions),
                        "THEN", conclusion)
                    facts.add(conclusion)
                    changed = True

    return facts

# -----
# Backward Chaining
# -----
def backward_chaining(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()

    if goal in facts:
        print("Fact found:", goal)
        return True

    if goal in visited:
        return False

    visited.add(goal)

    for rule in rules:
        if rule["conclusion"] == goal:
            conditions = rule["conditions"]

            print("Trying Rule: IF",
                " AND ".join(conditions),
                "THEN", goal)

            if all(backward_chaining(
                condition, facts, rules, visited)
                for condition in conditions):
                return True

    return False

# -----
# Main Program
# -----
print("=====")
print("    RULE-BASED EXPERT SYSTEM")
print("=====")

print("\nInitial Facts:")
for fact in facts:
    print("-", fact)

new_facts = forward_chaining(facts, rules)

print("\nFacts after Forward Chaining:")
for fact in new_facts:
    print("-", fact)

goal = "flu"

```

```

print("\nBACKWARD CHAINING")
print("-----")

result = backward_chaining(goal, facts, rules)

print("\nGoal:", goal)

if result:
    print("Result: Goal can be proved.")
else:
    print("Result: Goal cannot be proved.")

```

12. Sample Output

```

=====
      RULE-BASED EXPERT SYSTEM
=====

Initial Facts:
- fever
- cough

FORWARD CHAINING
-----
Rule Fired: IF fever AND cough THEN flu

Facts after Forward Chaining:
- fever
- cough
- flu

BACKWARD CHAINING
-----
Trying Rule: IF fever AND cough THEN flu
Fact found: fever
Fact found: cough

Goal: flu
Result: Goal can be proved.

```

13. Working of the Program

Forward Chaining: The system begins with fever and cough. Rule 1 is satisfied, so flu is added as a new fact.

Backward Chaining: The system begins with the goal flu. It finds Rule 1 and checks its conditions, fever and cough. Both are available, so the goal is proved.

14. Test Cases

Test Case 1: Facts = fever, cough; Goal = flu → Goal can be proved.

Test Case 2: Facts = fever; Goal = flu → Goal cannot be proved because cough is missing.

Test Case 3: Facts = fever, cough; Goal = infection → Goal cannot be proved because headache is missing.

15. Advantages

1. Easy to understand and implement.
2. Rules can be modified easily.
3. Suitable for decision-making systems.
4. Useful for diagnosis and troubleshooting.
5. Can represent expert knowledge.

16. Limitations

1. A large number of rules can make the system complex.
2. It depends on the quality of the rules.

3. It cannot automatically learn new rules.
4. Conflicting rules may create difficulties.
5. Real-world problems may require uncertainty or probabilistic reasoning.

17. Applications

Medical diagnosis; computer troubleshooting; network fault detection; banking; customer support; industrial fault diagnosis; recommendation systems; education; decision-support systems.

18. Result

A Rule-Based Expert System was successfully developed in Python using Forward Chaining and Backward Chaining inference mechanisms. Forward Chaining derived the conclusion from available facts, while Backward Chaining verified the goal by working backward through the rules.

19. Viva Questions

1. What is an Expert System?
2. What is a Rule-Based Expert System?
3. What is Forward Chaining?
4. What is Backward Chaining?
5. Which chaining method is data-driven?
6. Which chaining method is goal-driven?
7. What is a fact?
8. What is a rule?
9. What is an inference engine?
10. What is the main difference between Forward and Backward Chaining?